

Angular Learning Series

Angular Foundations -- Quick Overview

Updated for Modern Angular | input() output() inject() Signals Standalone

What you need to know before Topic 20

Contents

- F1. Angular CLI and Project Structure [UPDATED]
- F2. Angular Template Syntax -- The Four Bindings [Clean]
- F3. Built-in Control Flow -- @if, @for, @switch [Clean]
- F4. Component Lifecycle Hooks [UPDATED]
- F5. input() and output() -- Component Communication [UPDATED]
- FS. Foundation Summary -- The Mental Model [UPDATED]

Sections marked UPDATED have been rewritten to use modern Angular syntax: input() output() inject() standalone no NgModule no EventEmitter. Sections marked Clean were already using modern patterns and are unchanged.

F1. Angular CLI and Project Structure

The Command That Creates Everything

```
ng new my-app --standalone --routing --style=scss
```

```
cd my-app
```

```
ng serve # runs at localhost:4200
```

Note: --standalone is the default from Angular 17+. You do not need to pass it explicitly on newer versions -- a plain `ng new my-app` already generates a standalone project.

Folder Structure -- What Actually Matters

```
my-app/
```

```
src/
```

```
app/
```

```
app.component.ts (- root component
```

```
app.component.html (- root template
```

```
app.config.ts (- app-wide providers (standalone -- replaces AppModule)
```

```
app.routes.ts (- route definitions
```

```
main.ts (- bootstraps the app
```

```
styles.scss (- global styles
```

```
angular.json (- CLI configuration
```

```
tsconfig.json (- TypeScript config
```

There is no AppModule in a modern Angular project. `app.config.ts` is where all app-wide providers live. `main.ts` bootstraps directly with `bootstrapApplication()`.

CLI Commands You Will Use Daily

```
ng generate component features/users/user-list # create standalone component
```

```
ng generate service features/users/user # create service
```

```
ng generate pipe shared/pipes/currency # create pipe
```

```
ng build --configuration=production # production build
```

```
ng test # run tests
```

```
ng lint # lint the project
```

One Thing to Know

`main.ts` bootstraps the app with `bootstrapApplication(AppComponent, appConfig)`.

`appConfig` in `app.config.ts` is where you register all app-wide providers.

```
// app.config.ts -- the modern Angular entry point
```

```
import { ApplicationConfig } from '@angular/core';
```

```
import { provideRouter } from '@angular/router';
```

```
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';
export const appConfig: ApplicationConfig = {
```

```
  providers: [
    provideRouter(routes),
    provideHttpClient(),
  ]
};
```

// main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';
bootstrapApplication(AppComponent, appConfig)
```

```
.catch(err => console.error(err));
```

There is no AppModule, no BrowserModule.forRoot(), no NgModule anywhere.

F2. Angular Template Syntax -- The Four Bindings

Think of Angular templates as HTML with superpowers. Four binding types cover everything:

`{{ }}` -) Interpolation -) display a value

`[ ]` -) Property binding -) set a property FROM component

`( )` -) Event binding -) listen for an event, run component method

`[( )]` -) Two-way binding -) both directions at once (forms)

(!-- Interpolation -- display component data --)

`(h1){{ title }}/h1)`

`(p){{ user.name.toUpperCase() }}/p)`

(!-- Property binding -- set element property from component --)

`(img [src]="user.avatarUrl")`

`(button [disabled]="isLoading")Submit(/button)`

`(div [class]="cardClass")content(/div)`

(!-- Event binding -- run component method on event --)

`(button (click)="onSave()")Save(/button)`

`(input (input)="onSearch($event)")`

`(form (submit)="onSubmit()")`

(!-- Two-way binding -- for forms with NgModel --)

`(input [(ngModel)]="searchTerm")`

(!-- searchTerm updates when user types

input updates when searchTerm changes in component --)

(!-- Combining them -- very common pattern --)

`(button`

`[disabled]="!form.valid || isLoading"`

`(click)="onSubmit()")`

`{{ isLoading ? 'Saving...' : 'Save' }}`

`(/button)`

F3. Built-in Control Flow -- @if, @for, @switch

Angular 17+ replaced ngIf and ngFor with cleaner block syntax.

You will see both in codebases -- know both.

(!-- @if -- show/hide blocks --)

```
@if (isLoading) {  
(div class="spinner")Loading...(div)  
} @else if (error) {  
(div class="error"){{ error }}(div)  
} @else {  
(div class="content"){{ data }}(div)  
}
```

(!-- Old syntax -- still common in existing codebases --)

```
(div *ngIf="isLoading; else content")Loading...(div)  
(ng-template #content)(div){{ data }}(div)(ng-template)
```

(!-- @for -- render a list --)

```
@for (product of products; track product.id) {  
(app-product-card [product]="product" /)  
}  
  
@empty {  
(p)No products found(/p)  
}
```

(!-- Old syntax --)

```
(app-product-card  
*ngFor="let product of products; trackBy: trackById"  
[product]="product")  
(/app-product-card)  
  
(!-- @switch -- multiple conditions --)  
@switch (user.role) {  
  @case ('admin') { (app-admin-panel /) }  
  @case ('editor') { (app-editor-panel /) }  
  @default { (app-user-panel /) }  
}
```

track in @for -- Why It Matters

track product.id tells Angular which property uniquely identifies each item.

Without it Angular re-renders the entire list on every change.

With it Angular only updates what actually changed.

Always track by a unique id.

F4. Component Lifecycle Hooks

Think of a component's life like a person's day:

Born -) constructor (do not do much here -- use inject() here only)

Briefed -) ngOnChanges (inputs arrived)

Starts work -) ngOnInit (data loading, subscriptions)

Checks the clock -) Angular's change detection runs

Sees the room -) ngAfterViewInit (DOM is ready)

Goes home -) ngOnDestroy (cleanup)

// Modern Angular -- inject() replaces constructor injection

// input() replaces @Input() decorator

```
import { Component, OnInit, OnChanges, AfterViewInit,
OnDestroy, SimpleChanges, inject, input } from '@angular/core';
```

```
import { UserService } from './user.service';
```

```
@Component({ selector: 'app-user', standalone: true, template: '...' })
```

```
export class UserComponent implements OnInit, OnChanges, AfterViewInit, OnDestroy {
```

```
    // input() signal -- modern replacement for @Input() decorator

    userId = input.required(number)();
```

```
    // inject() -- modern replacement for constructor injection

    private readonly userService = inject(UserService);
```

```
    // Runs when input() signal values change

    // Runs BEFORE ngOnInit on first load

    ngOnChanges(changes: SimpleChanges): void {

        if (changes['userId']) {

            console.log('userId changed:', changes['userId'].currentValue);

        }

    }
```

```
    // Runs ONCE after first ngOnChanges

    // Best place for: HTTP calls, subscriptions, setup

    ngOnInit(): void {

        this.userService.loadUser(this.userId());

    }
```

```
// Runs after the template and child components are rendered

// Best place for: viewChild() access, DOM manipulation

ngAfterViewInit(): void {

// this.inputRef() replaces @ViewChild -- see Topic 32

}
```

```
// Runs just before component is destroyed

// Best place for: unsubscribe, cleanup, cancel timers

// With takeUntilDestroyed/DestroyRef you rarely need this manually

ngOnDestroy(): void {

// cleanup if not using DestroyRef

}

}
```

The 80% Rule

ngOnInit and ngOnDestroy cover 80% of all lifecycle needs:

ngOnInit = start the work (load data, set up subscriptions)

ngOnDestroy = clean up (unsubscribe, cancel timers)

In modern Angular with takeUntilDestroyed and DestroyRef, you often do not need ngOnDestroy at all for subscription cleanup. See Topic 21 -- takeUntilDestroyed and DestroyRef.

F5. input() and output() -- Component Communication (Modern Angular)

The One-Way Street Rule

Angular data flows down through input() and events bubble up through output().

Instructions go down, reports come up.

Parent Component

```
[user]="selectedUser" -) input() -- data goes DOWN
```

```
(userSelected)="..." -) output() -- events go UP
```

Child Component

```
// ---- CHILD COMPONENT -- modern Angular 17+ syntax ----
```

```
import { Component, input, output } from '@angular/core';
```

```
import { User } from './user.model';
```

```
@Component({
```

```
  selector: 'app-user-card',
```

```
  standalone: true,
```

```
  template: `
```

```
    (div class="card")
```

```
      (h3){ { user().name } }(/h3)
```

```
      (button (click)="onSelect()")Select(/button)
```

```
      (button (click)="onDelete()")Delete(/button)
```

```
    (/div)
```

```
  `
```

```
})
```

```
export class UserCardComponent {
```

```
  // input() -- receives data FROM parent
```

```
  // .required() means parent MUST provide this value
```

```
  user = input.required(User)();
```

```
  isSelected = input(false); // optional input with default
```

```
  // output() -- sends events TO parent
```

```
  // No EventEmitter needed -- output() handles it
```

```
  userSelected = output(User)();
```

```
  userDeleted = output(number)(); // emits the id
```

```
  onSelect(): void { this.userSelected.emit(this.user()); }
```

```

    onDelete(): void { this.userDeleted.emit(this.user().id); }

}

```

// ---- PARENT COMPONENT ----

```

import { Component, signal } from '@angular/core';
import { UserCardComponent } from './user-card.component';
import { User } from './user.model';
@Component({
  selector: 'app-user-list',
  standalone: true,

```

```

  imports: [UserCardComponent], // no CommonModule needed

  template: `
    @for (user of users(); track user.id) {

      (app-user-card

        [user]="user"

        [isSelected]="selectedId() === user.id"

        (userSelected)="onUserSelected($event)"

        (userDeleted)="onUserDeleted($event)" /)

    }
    `
  })
  export class UserListComponent {

    users = input.required(User[])([]);

    selectedId = signal(number | null)(null);

```

```

    onUserSelected(user: User): void { this.selectedId.set(user.id); }

    onUserDeleted(id: number): void { /* emit further up if needed */ }

  }

```

New Angular 17+ Syntax -- input() and output() functions

input() and output() are signal-based functions that replace the decorator approach.

```

Old way (still valid but being phased out):

@Input({ required: true }) user!: User;

@Output() userSelected = new EventEmitter<User>();

```

New way (preferred):

```
user = input.required(User());
```

```
userSelected = output(User());
```

Key differences:

1. `input()` returns a Signal -- read the value with `user()` not `user`
2. `output()` requires no `EventEmitter` -- just call `.emit()` directly
3. No decorators needed -- fewer imports, cleaner code
4. TypeScript infers types automatically from the generic parameter
5. `input.required()` causes a compile error if parent does not provide the value

Foundation Summary -- The Mental Model

Angular App

main.ts -) bootstrapApplication(AppComponent, appConfig)

app.config.ts -) provideRouter(), provideHttpClient()

AppComponent (root -- always visible)

template uses {{ }}, [], (), [()]

@if / @for / @switch for logic

ChildComponent A

receives data via input()

sends events via output()

ChildComponent B

same pattern

Services (shared logic, injected via inject())

@Injectable({ providedIn: 'root' })

Four things every Angular component does:

1. Receives data -- via input() from parent
2. Displays data -- via template ({{ }}, @if, @for)
3. Reacts to events -- via (click), (input) etc.
4. Communicates up -- via output() to parent

Modern Angular Patterns at a Glance

Instead of Use this

@Input() user: User user = input.required(User())

@Output() ev = new EventEmitter(X)() ev = output(X)()

constructor(private svc: S) private svc = inject(S)

@ViewChild(EI) el: EI el = viewChild(EI)(EI)

CommonModule in imports specific imports: AsyncPipe, NgIf etc.

AppModule / NgModule standalone: true + app.config.ts

Reading input signal values: this.user() with parentheses -- it is a signal

Reading output: nothing to read -- just call this.ev.emit(value)

The One Mental Model to Remember

Every Angular app is a tree of standalone components.

Data flows DOWN via input() -- events flow UP via output().

Services are shared logic injected anywhere via inject().

The router decides which components appear at which URLs.

Signals replace zone.js -- when a signal changes, only the components that read it update -- nothing else re-renders.